Process Scheduling
Part I

Sistemas de Computadores

Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Overview

- Main concepts
- Non-preemptive scheduling
- Preemptive scheduling

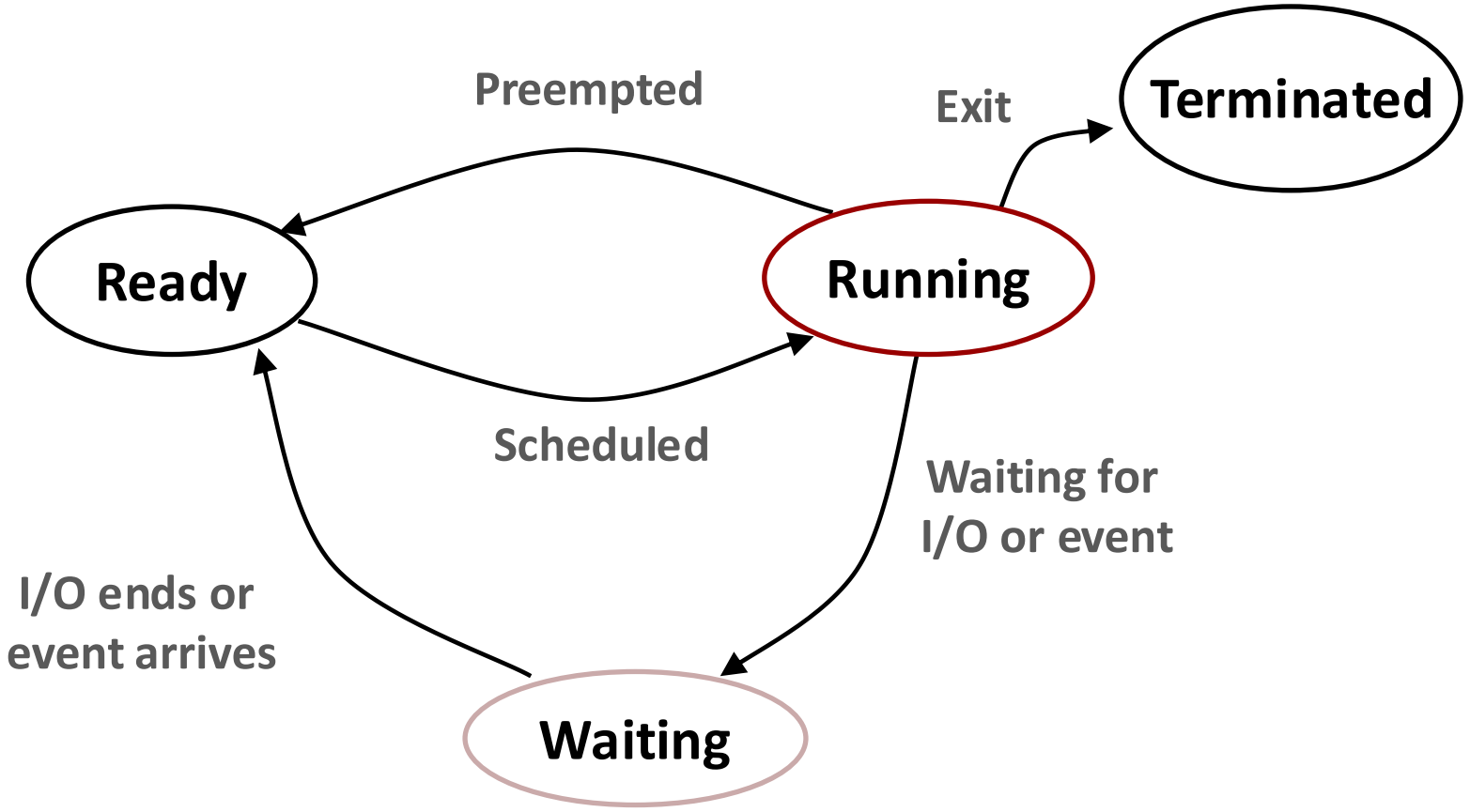
Multitasking

- **Modern OSes can run several processes at the same time**

- **On a single-core system, this is only an illusion**
  - The OS rapidly switches the CPU between processes, creating the perception of parallel execution

- **On multicore systems, true parallelism is possible**
  - The OS schedules processes across multiple cores to maximize utilization
  - However, scheduling issues become correspondingly more complex due to issues like load balancing, cache affinity, and synchronization across cores

Process states



Process types

- **I/O-bound**
  - Frequent I/O requests → frequent blocking
  - Short CPU bursts, alternating with waiting periods
  - Limited continuous CPU usage
  - Common in interactive and I/O-intensive applications

- **CPU-bound**
  - Intensive computation with little I/O
  - Rarely block → long CPU bursts
  - Tend to run until preempted or completion
  - Common in compute-intensive workloads

Multitasking

- **How is CPU time allocated to runnable processes?**

- **Only runnable processes are considered for CPU execution**
  - Other states (e.g., blocked/waiting) cannot run even if CPU is idle
  - These processes are ignored by the scheduler

- **This leads to two core responsibilities of the kernel:**
  - **Scheduling**: Deciding which process runs next and for how long
  - **Dispatching**: Performing the context switch to start running the selected process

Dispatcher

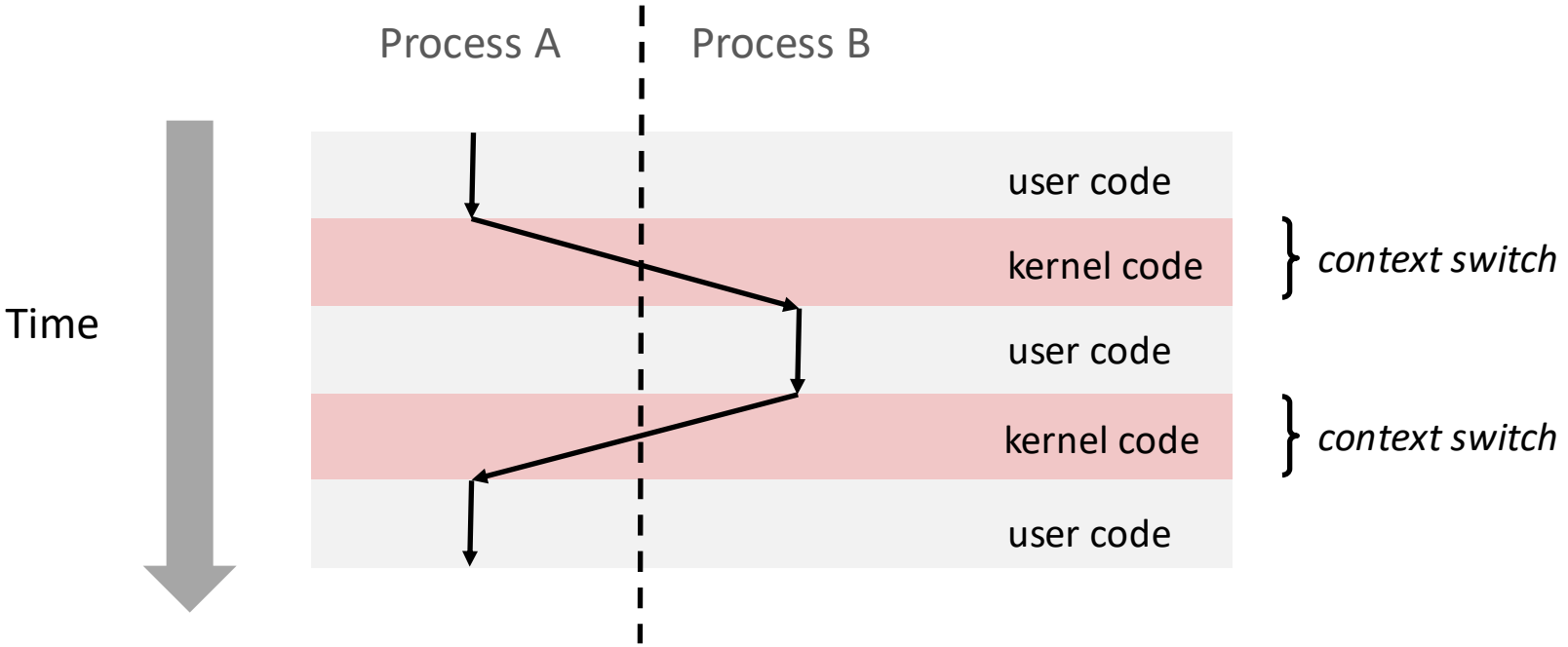
- **The dispatcher transfers control of the CPU to the process selected by the scheduler**

- **As part of this operation, it performs a context switch:**
  - Save the state of the current process
  - Restore the state of the selected process
  - Switch to user mode
  - Resume execution at the correct instruction

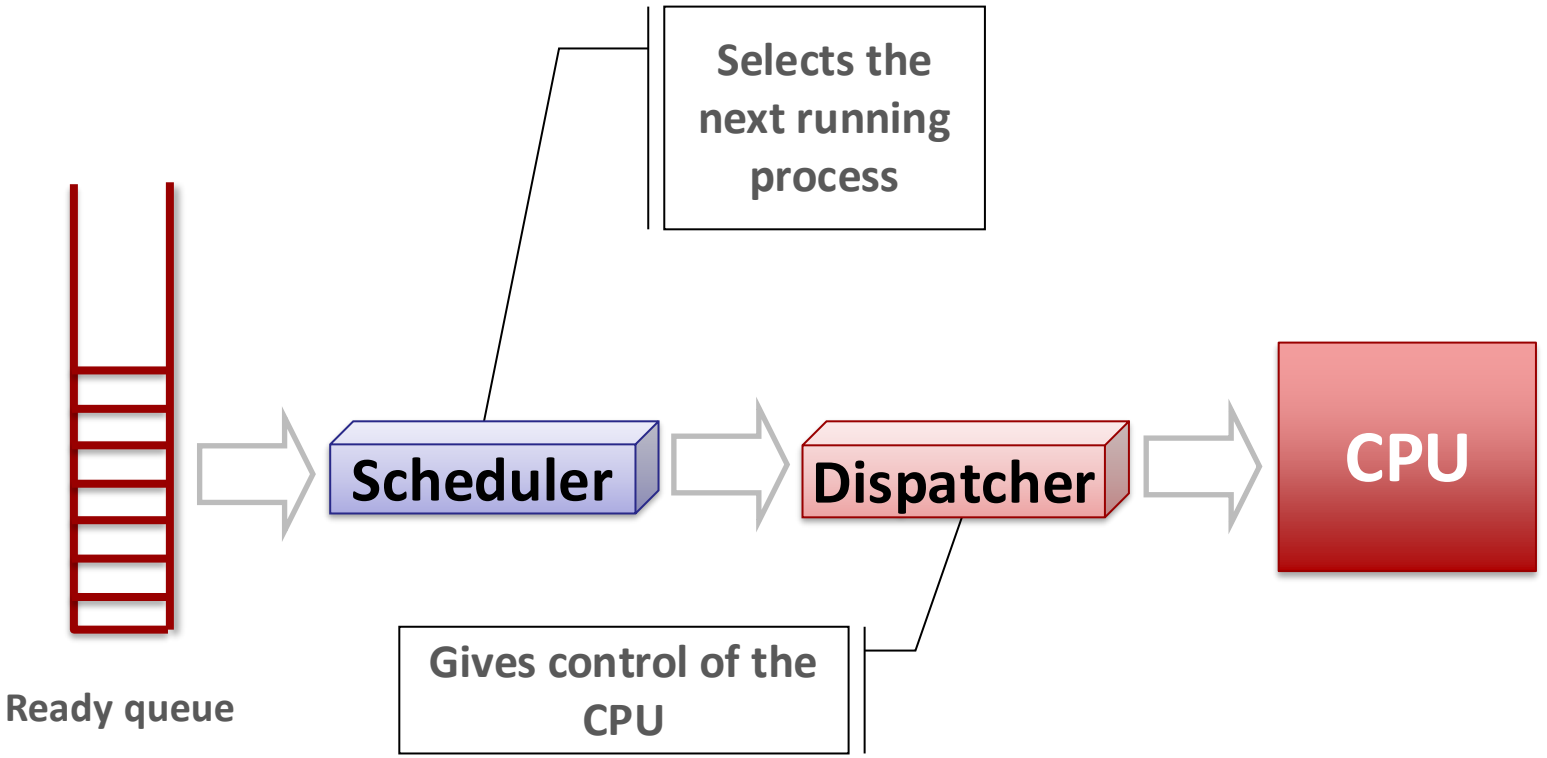
- **Efficiency is critical:**
  - Invoked on every process switch
  - Must minimize overhead to avoid degrading system performance

Context switching

- **Control flow passes from one process to another via a context switch**
  - The context is represented in the PCB of the process



Scheduler and Dispatcher



Process scheduling

- **Determines which process runs, when, and for how long**
  - Fundamental component of a multitasking operating system
- **Responsible for efficient CPU utilization:**
  - Selects the next process to execute
  - Allocates CPU time among runnable processes
- **Scheduling operates at different levels:**
  - **Short-term** (CPU scheduler) → selects the next process to run
    - (focus of this course, since it is used on all OSes)
  - **Medium- and long-term** schedulers → manage process admission and suspension

Scheduling queues

- **The OS maintains collections of processes, typically implemented as queues**
  - Often represented as doubly-linked lists of PCB pointers
- **In practice, schedulers use more advanced data structures to improve efficiency and fairness:**
  - Priority queues / heaps
  - Red–black trees
  - Multilevel feedback queues
- **The run queue (or ready queue) contains all processes that are ready to execute but not currently running on a CPU**

Scheduling queues

- **When a process issues a long-running request (e.g., disk I/O):**
  - It cannot continue executing
  - It is removed from the run queue and moved to the blocked (waiting) state
- **Blocked processes are organized into per-resource wait queues**
  - Not a single global queue
  - Each process waits only for the specific resource or event it depends on
- **Kernel actions:**
  - Enqueue the process in the appropriate wait queue
  - Select and schedule another ready process

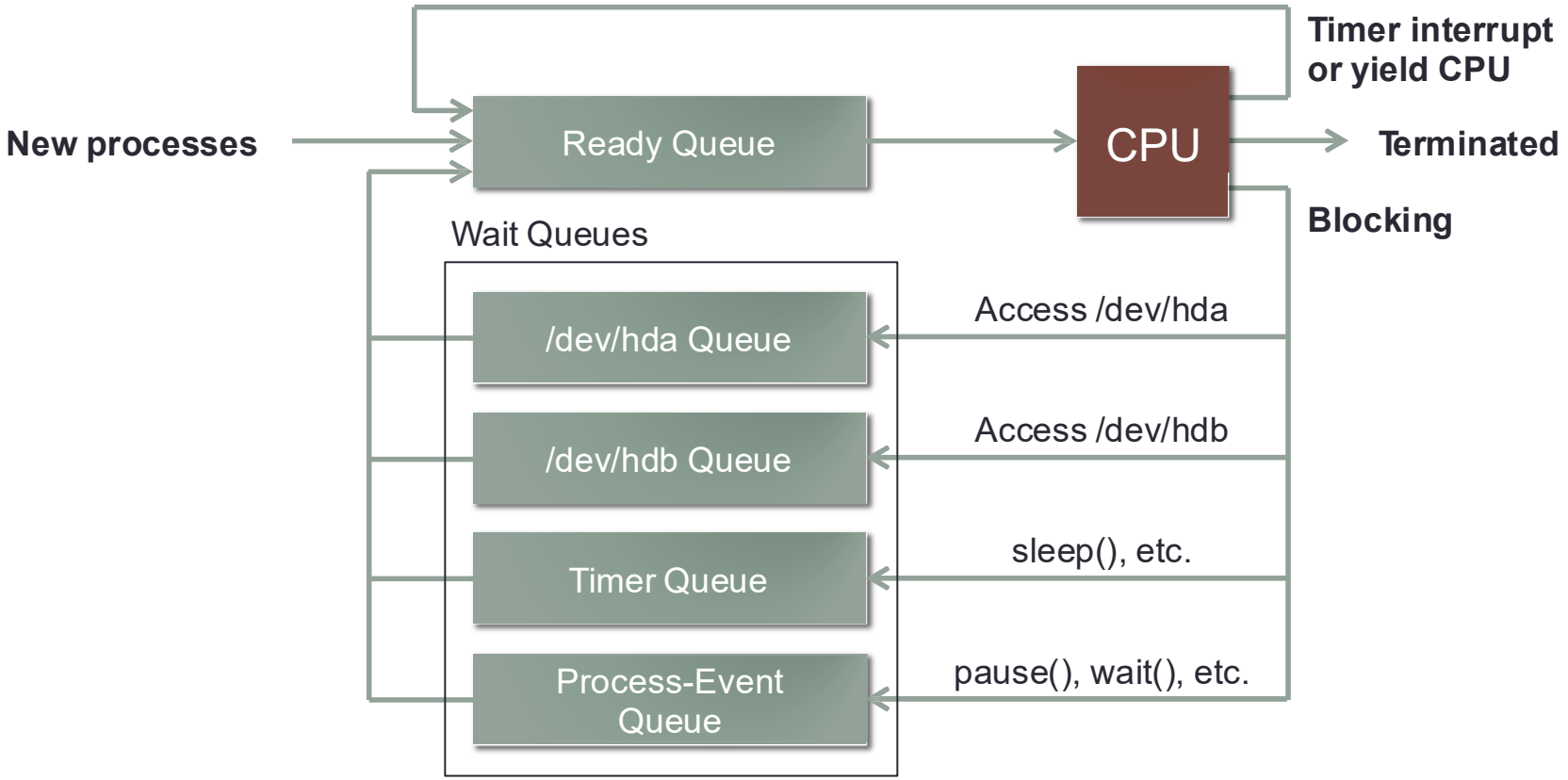
Scheduling queues

- **When a device signals completion (via an interrupt):**
  - The interrupt handler identifies the relevant wait queue
  - Dequeues the waiting process(es)
  - Marks them as ready (runnable) and moves them to the run queue

- **Wait queues are a general mechanism, not limited to hardware**
  - Used for any blocking condition
  - E.g., I/O completion, signals, child process termination, condition-based synchronization

Process scheduling: “queuing diagram”

- Example “queuing diagram” of ready queue and wait queues



When does scheduling happen?

- **Scheduling decisions may occur under the following conditions:**
  1. Running → Waiting
     - Process blocks (e.g., I/O request, waiting for child)
  2. Running → Ready
     - Process is preempted (e.g., time slice expires)
  3. Waiting → Ready
     - Event completes (e.g., I/O finishes)
  4. Process terminates

- **Preemption considerations:**
  - Cases 1 and 4: no preemption required
  - Cases 2 and 3: may involve preemption, depending on the scheduling policy

Scheduling algorithms

- **Real-world OSes use hybrid and adaptive scheduling algorithms**
  - Combine multiple strategies (e.g., priority + time slicing)
  - Dynamically adjust behavior based on system state
- **Algorithm design depends on workload characteristics**
  - CPU-bound vs I/O-bound processes
  - Interactive vs batch workloads
  - Short vs long-running tasks
- **No single algorithm performs best in all scenarios**

Scheduling metrics

- **To compare scheduling algorithms, we use metrics that capture efficiency, responsiveness, and fairness**
  - Scheduling decisions involve trade-offs, so metrics **must be interpreted collectively** rather than in isolation

- **CPU utilization**
  - Fraction of time the CPU is busy
  - Can be estimated using tools such as `top`, `htop`, or profilers

- **Throughput**
  - Number of processes completed per unit of time

Scheduling metrics

- **Turnaround time**
  - Total time from submission to completion
  - Includes execution, waiting, and I/O time (wall-clock time)
- **Waiting time**
  - Total time spent in the ready queue
  - Time the process is ready but not executing
- **Response time**
  - Time from submission to first response
  - Measures responsiveness, not completion time

Metrics perspective

- **Average values**
  - Reflect overall system performance across all processes (e.g., average waiting time)
- **Minimum / Maximum values**
  - Capture best- and worst-case behavior, important for guarantees and real-time constraints
- **Variance**
  - Measures consistency and predictability
  - Low variance indicates a more stable and fair scheduling behavior

Scheduling goals

- **Maximize CPU utilization and throughput**
  - Ensure the processor is kept busy and as many processes as possible are completed per unit of time
- **Minimize turnaround time, waiting time, and response time**
  - Improve both overall efficiency (batch workloads) and responsiveness (interactive workloads)
- **Ensure fairness among processes**
  - Prevent starvation and guarantee that all processes receive a reasonable share of CPU time

Scheduling trade-offs

- **Different scheduling algorithms optimize different aspects of system performance**
  - May favor CPU-bound processes (long CPU bursts) or I/O-bound processes (short, frequent bursts)
- **Impact key metrics differently**
  - Throughput (number of completed processes)
  - Latency / response time (how quickly a process reacts)
  - Fairness (how evenly CPU time is distributed)
- **Can introduce undesirable effects**
  - Starvation: some processes may never get CPU time
  - Convoy effect: short processes wait behind long ones, degrading performance

Key design considerations

- **Preemptive vs non-preemptive scheduling**
  - Determines whether running processes can be interrupted
- **Time slice (quantum) size**
  - Affects responsiveness vs overhead trade-off
- **Priority management and aging**
  - Controls process importance and mitigates starvation
- **Context-switch overhead**
  - Frequent switching improves responsiveness but incurs performance cost

Choosing a scheduling algorithm

- **Must align with system goals:**
  - Responsiveness (interactive systems)
  - Throughput (batch systems)
  - Fairness (multi-user environments)

- **Requires understanding of:**
  - Workload characteristics (CPU-bound vs I/O-bound, interactive vs batch)
  - Algorithm properties and limitations

Graphical notation

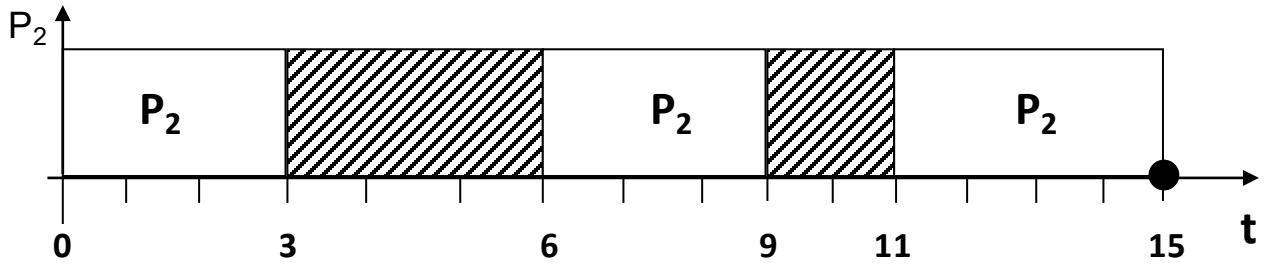
Process arrives

Process is running

Process is waiting

Process ends

Example



Overview

- Main concepts
- **Non-preemptive scheduling**
- Preemptive scheduling

Non-preemptive scheduling

- **Once a process enters the running state, it continues executing:**
  - Until it completes, or
  - Until it voluntarily yields (e.g., blocks for I/O)

- **Scheduling decisions occur only when:**
  - The process terminates, or
  - The process blocks

- **Key characteristic:**
  - No preemption — the OS does not forcibly interrupt a running process
  - Context switches occur only at these explicit hand-off points

First-In, First-Out

- **Also known as First Come, First Served (FCFS), is the simplest scheduling algorithm**
  - It simply selects processes in the order that they arrive in the ready queue

+ Scheduling overhead is minimal

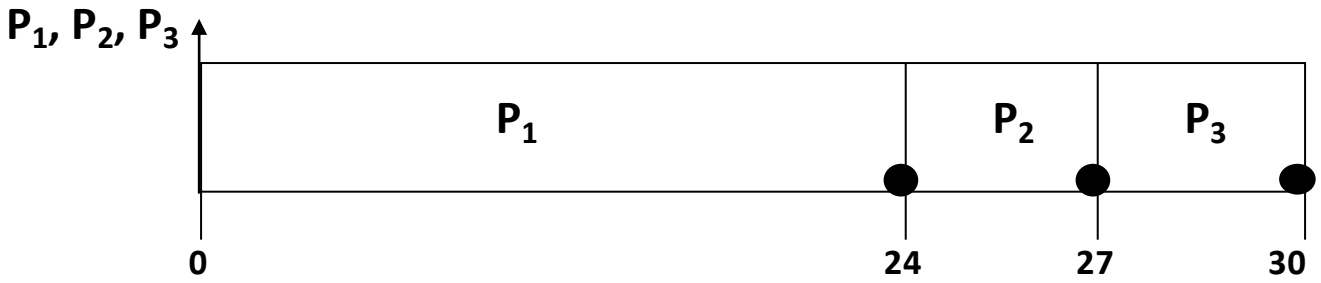
- Context switches only occur upon process termination or blocking, and no reorganization of the process queue is required

– However, next examples demonstrate that:

- Throughput can be low, since long processes can hog the CPU
- The average waiting time may vary substantially if the processes' execution times vary greatly

First-In, First-Out

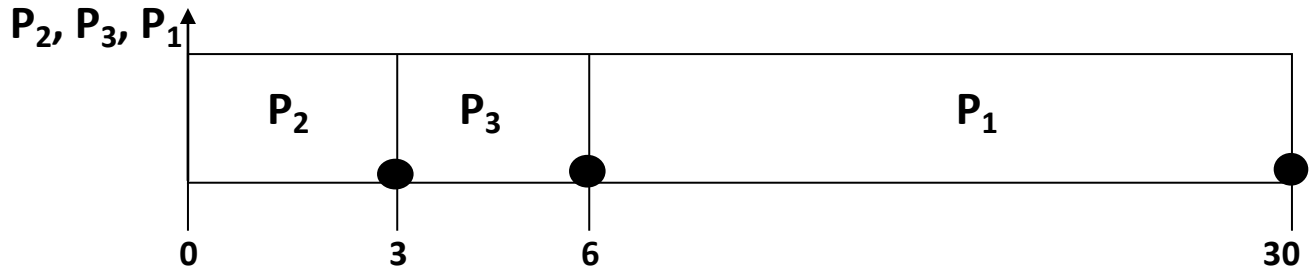
Arrival order: {P_1, P_2, P_3}



Process	Execution time	Turnaround time
P_1	24	24
P_2	3	27
P_3	3	30
		Average = 27

First-In, First-Out

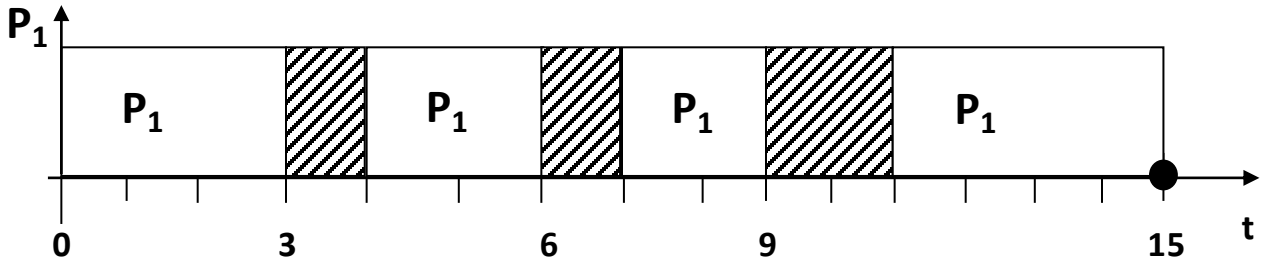
Arrival order: {P_2, P_3, P_1}



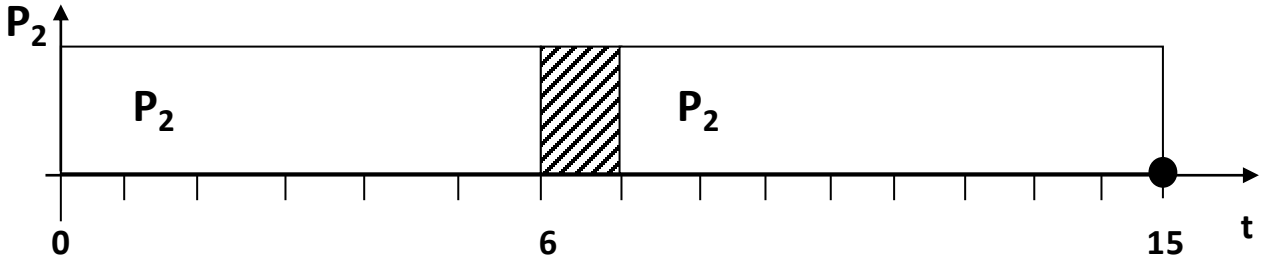
Process	Execution time	Turnaround time
P_1	24	30
P_2	3	3
P_3	3	6
		Average = 13

FIFO with I/O-bound and CPU-bound

- Execution profile of P_1 (I/O-bound)

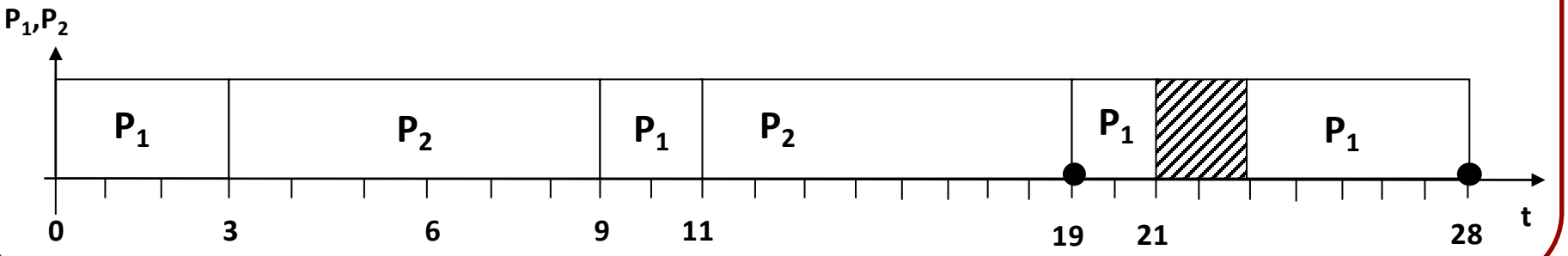


- Execution profile of P_2 (CPU-bound)

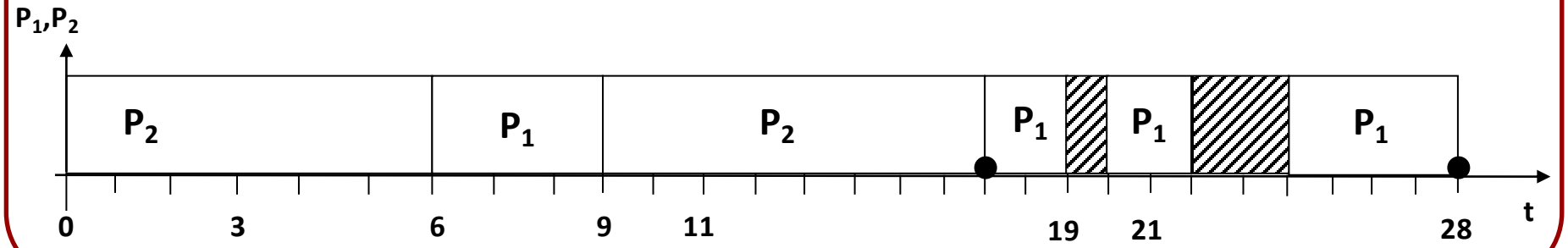


FIFO with I/O-bound and CPU-bound

Arrival order: {P_1, P_2}



Arrival order: {P_2, P_1}



FIFO with I/O-bound and CPU-bound

- **Bias towards CPU-bound processes**
  - CPU-bound processes tend to dominate CPU usage
  - They hold the CPU for long periods and delay other processes
- **Impact on I/O-bound processes**
  - When an I/O-bound process requests I/O, it releases the CPU and moves to a wait queue
  - After the I/O completes, it returns to the end of the ready queue
  - It must then wait for all preceding processes to execute
  - This leads to poor responsiveness for I/O-bound (often interactive) tasks

FIFO with I/O-bound and CPU-bound

- Can lead to the convoy effect
  - Short I/O-bound processes are delayed behind long CPU-bound ones

- Key takeaway:
  - Best suited for batch workloads
  - Inefficient for interactive systems requiring quick response times

Overview

- Main concepts
- Non-preemptive scheduling
- **Preemptive scheduling**

Preemptive scheduling

- **The OS can interrupt a running process without its cooperation**
  - The process is paused and can be resumed later
  - Triggered by hardware interrupts (e.g., timer interrupt)

- **Supported by nearly all modern OSes**
  - Suspends the current process
  - Performs a context switch
  - Invokes the scheduler to select the next process

- **Why it matters**
  - Enables fair sharing of the CPU
  - Improves responsiveness for interactive processes

Time slice

- **A time slice (quantum) is the maximum amount of CPU time a process can use before being preempted**

- **The scheduler must define an appropriate quantum size**
  - A key design parameter with significant impact on performance
  - Choosing the right quantum balances responsiveness and efficiency

- **Too large:**
  - Poor responsiveness for interactive processes
  - Behavior approaches non-preemptive (FIFO) scheduling

- **Too small:**
  - High context-switch overhead
  - Reduced effective CPU utilization

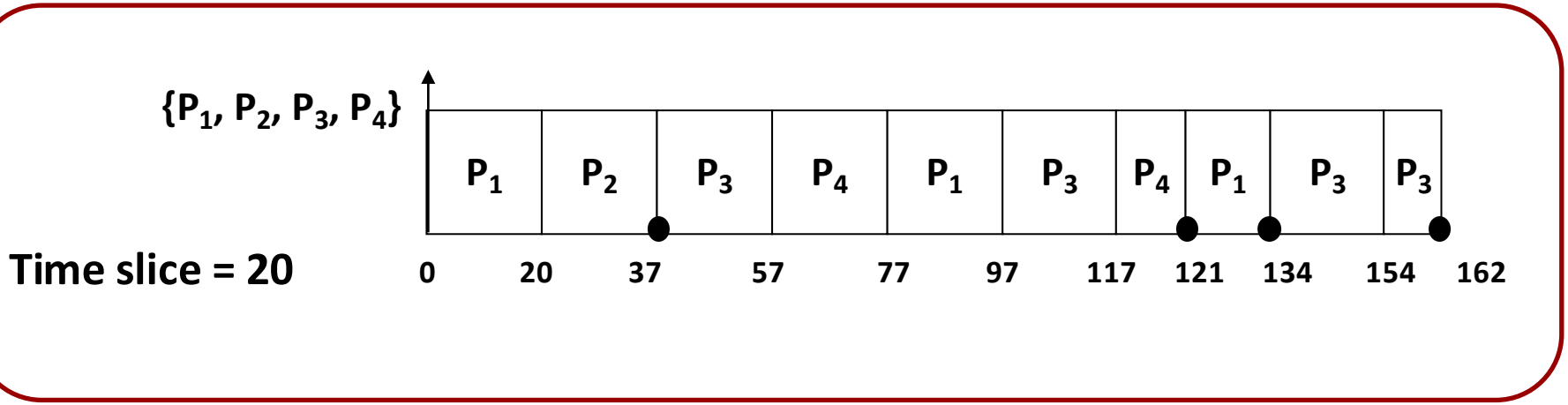
Round-Robin

- **Processes are scheduled in a fair and cyclic manner**
  - Each process is assigned an **equal time slice** (quantum)
  - Processes are executed in **circular** (round-robin) **order**, ensuring regular CPU access for all

- **Execution model:**
  - A running process may be preempted when its time slice expires
  - It is then placed at the end of the ready queue

- **Scheduling decisions occur when:**
  - The time slice expires (preemption)
  - The process blocks (e.g., I/O)
  - The process terminates

Round-Robin



Process	Execution time
P_1	53
P_2	17
P_3	68
P_4	24

Round-Robin

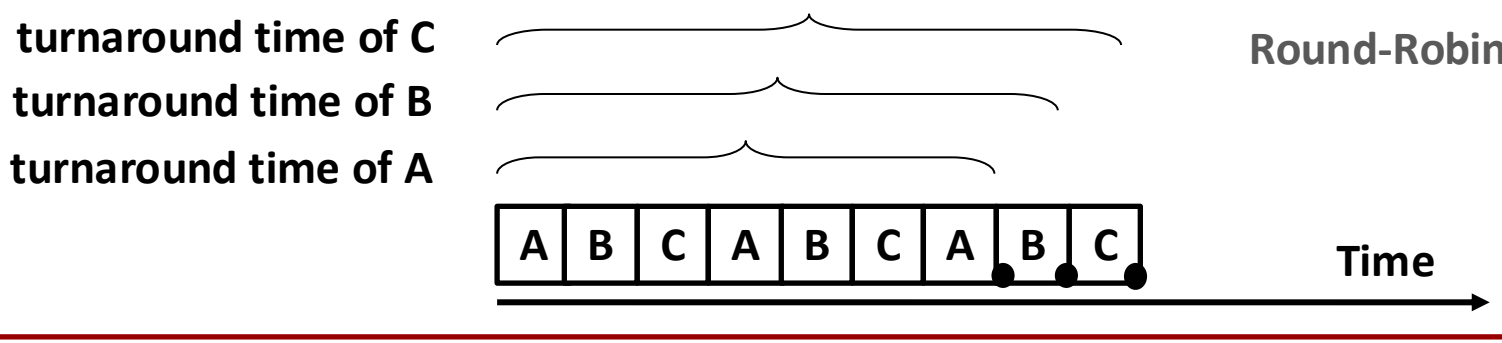
+ **RR scheduling is simple, easy to implement, and starvation-free**
  - Each process must wait no longer than $(n - 1) \times q$ time units until its next time quantum
  - Responsiveness: interactive tasks get regular CPU access

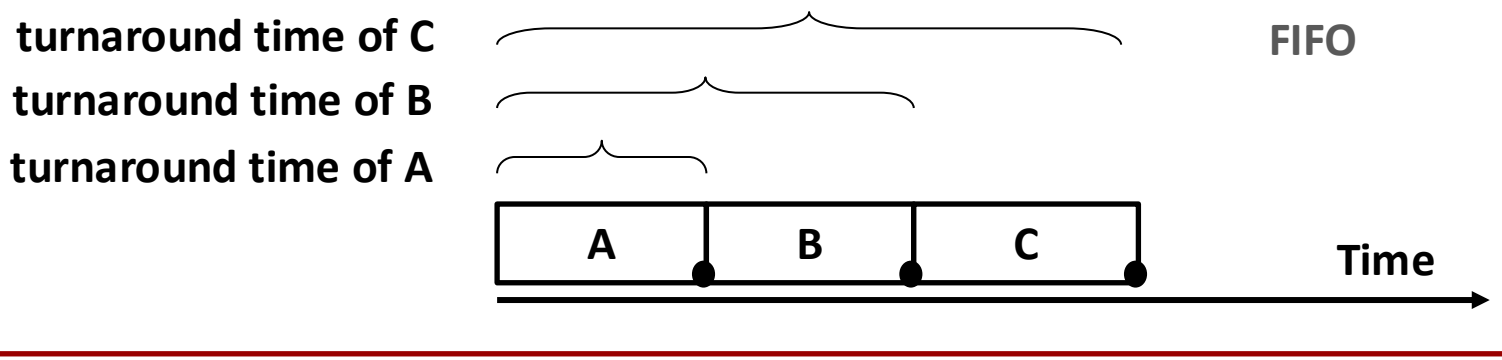
– **Performance depends heavily on the size of the time slice**
  - If time slice is too long, scheduling degrades to FIFO
  - If time slice is too short, throughput suffers and context switching cost dominates

■ **In practice, modern OSes use time slices on the order of tens of milliseconds**
  - The context-switch time is typically much smaller (microseconds), making it only a small fraction of the time slice

FIFO vs Round-Robin

- **With zero-cost context switch, is RR always better than FIFO?**
  - Consider the following three jobs of equal execution times





FIFO vs Round-Robin

- **RR guarantees fairness – no one starves – but can be uniformly inefficient under certain workloads**
  - May be less efficient than simpler policies (e.g., FIFO)

- **Every process is treated with identical priority**
  - Each gets the same fixed CPU quantum
  - If all jobs have similar lengths, RR's frequent context-switches inflate average turnaround

- **Next lecture we'll look at priority-based scheduling**
  - Where the relative importance of each process may be precisely defined

Linux real-time scheduling policies

- **SCHED_FIFO (First In, First Out)**
  - Priority-based scheduling algorithm
  - Tasks run until they voluntarily yield CPU or are preempted by higher priority tasks
  - Risk of lower priority tasks being starved

- **SCHED_RR (Round Robin)**
  - Priority-based scheduling with time slicing
  - Tasks run for a time quantum (time slice) before being preempted
  - Ensures fairness among tasks of the same priority

- **Review the Moodle example code carefully to understand how these policies behave in real scheduling scenarios**

Summary

- **Core concepts**
  - The scheduler decides which process runs next and for how long
  - The dispatcher performs the context switch and transfers CPU control to the selected process

- **Scheduling goals**
  - Maximize CPU utilization and system throughput
  - Minimize turnaround time, waiting time, and response time
  - Balance average values, extremes (min/max), and variance to ensure fairness and predictability

- **Algorithms discussed**
  - Non-preemptive: FCFS – simple, but may lead to long wait times
  - Preemptive: Round-Robin – fair and responsive, depends on time quantum